JULIO GARAVITO COLOMBIAN SCHOOL OF ENGINEERING



UNIVERSIDAD

Assignment:

LAB 4 – Thread Modeling

Teacher:

Ing. Daniel Vela

Members:

Ivan Santiago Forero Torres

Juan Sebastián Beltran Rodríguez

Nicolas Cuervo Naranjo

Systems Engineering

Bogotá D.C

16/2/26

Contents

Introduction. 3
Methodology. 4
Executive Summary. 4
Technical Summary. 4
Scope of Analysis. 5
Limitations of the Study 5
Development: 7
Part 1: Case introduction and asset analysis. 7
Part 2: Applying STRIDE to the DFD. 8
Part 3: Risk analysis with DREAD. 11
Part 4: Documentation and export. 14
Conclusions: 15

Introduction.

The digital transformation of financial services has shifted risk from the physical to the logical environment, making online banking platforms the most critical asset and, at the same time, the most exposed asset of an entity. This report details the threat modeling process performed on the banking infrastructure of a medium-sized bank.

The main objective was to identify, classify and prioritize the risks present in the interaction between the five core components of the system: the user interface (Frontend), the business logic (API Backend), the data storage (Database), the external communication (Email Service) and the administrative control (Admin Portal).

Scope of Analysis and Key Findings

During the exercise, the flow of data from when the client logs in using JWT tokens to when interbank transfers are executed through external processors was explored. The most relevant findings are summarized in three pillars:

1. **Critical Vulnerability in Transaction Logic:** The most alarming finding after applying the DREAD model was the possibility of Tampering in the Transaction API. The lack of integrity validation mechanisms (such as digital signatures) would allow an attacker to alter amounts or divert funds, obtaining a direct financial impact with moderate technical ease.
2. **Weakness in Non-Repudiation and Auditing:** A structural risk of Repudiation was identified because audit records are not immutable. This means that an attacker, especially an internal one (Administrator), could perform fraudulent actions and eliminate the forensic trail, leaving the bank without legal response or insurance recovery capabilities.
3. **Fragmentation in Communication Channel Trust:** Analysis of entry points revealed that the email service is a weak link. Without strict email authentication policies, the system is susceptible to spoofing, facilitating phishing campaigns that compromise the identity of customers.

Methodology.

For this analysis, two of the most recognized frameworks in the cybersecurity industry have been used:

- STRIDE: Used for the identification of threats in each component of the system by analyzing its data stream. Six categories are evaluated: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege.
- DREAD: Used to qualify and prioritize identified threats based on five vectors: Damage, Reproducibility, Exploitability, Affected Users, and Discoverability.
- DFD (Data Flow Diagram): A data flow diagram was developed to visualize the interactions between processes, data warehouses, and external entities.

Executive Summary.

The analysis determined that the system possesses a critical attack surface, mainly in the Backend API and transaction management. The most severe risk identified is transaction manipulation, with a rating of 4.2/5 (Critical). A structural weakness was detected in the management of logs, which are not immutable, facilitating the repudiation of malicious actions. Prioritizing the integrity of financial transactions and the protection of the user's identity.

Technical Summary.

Technical analysis of the banking ecosystem reveals an architecture with critical deficiencies in integrity validation and forensic evidence persistence. It was identified that, although the system uses HTTPS and JWT, the lack of encryption in the tokens and the persistence of optional 2FA generate a high surface of exposure to session hijacking and identity theft. The most severe finding lies in the Transactions API, which presents a Tampering vulnerability by lacking end-to-end digital signatures; this would allow an attacker to manipulate sensitive parameters in JSON requests (such as amounts or target accounts) without invalidating the authenticity of the session token.

At the same time, the logging infrastructure suffers from a structural vulnerability of Repudiation, since the audit logs are not immutable or centralized in an external environment, allowing actors with administrative privileges to alter or eliminate the trail of fraudulent activities. Finally, the weakness in the authentication protocols of the SPF

mail service and the absence of forced TLS in the SMTP transport degrade the trust of the communication channel, facilitating interception attacks and manipulation of security alerts. As a corrective measure, it is imperative to transition to a WORM-like log architecture and strengthen the API by implementing asymmetric signatures and strict server-side schema validation.

Scope of Analysis.

The threat modeling analysis focused exclusively on the logical ecosystem of the online banking platform described in the base case. The scope included:

- **Critical Data Flows:** From user authentication (Login/JWT) to the execution of transfers and the output of email notifications.
- **Interactions between Components:** Evaluation of the trust boundaries between the Frontend, the Backend API, the Database and the Administrative Portal.
- **Attack Vectors:** Identification of threats from both external actors (hackers) and internal actors (privileged administrators).
- **Integration with Third Parties:** Analysis of the connection point with the external payment processor, evaluating the risk of impersonation and manipulation in the exchange of messages.

Limitations of the Study

As cybersecurity engineers, it is vital to state that this study has methodological boundaries that should be considered for future audits:

- **Theoretical Nature:** This analysis was based on the logical design of the system (DFD) and not on execution tests. No penetration testing, dynamic vulnerability scanning (DAST), or source code review (SAST) were performed.
- **Physical Infrastructure Exclusion:** The study does not cover the physical security of data centers, nor network-level security (firewall configuration, VLAN segmentation, or protection against volumetric DDoS attacks).
- **Endpoint Security:** The integrity of the customers' end devices (mobile or computer) was not evaluated, assuming that the browser is the only point of interaction under relative control.

- **External Regulatory Compliance:** The analysis does not delve into local or international legal regulations (such as PCI-DSS or GDPR) beyond general information security best practices.
- **Third-Party Dependency:** The internal security of the third-party payment processor is considered out of scope; only the communication interface (API) that binds them to the bank was analyzed.

Development:

Part 1: Case introduction and asset analysis.

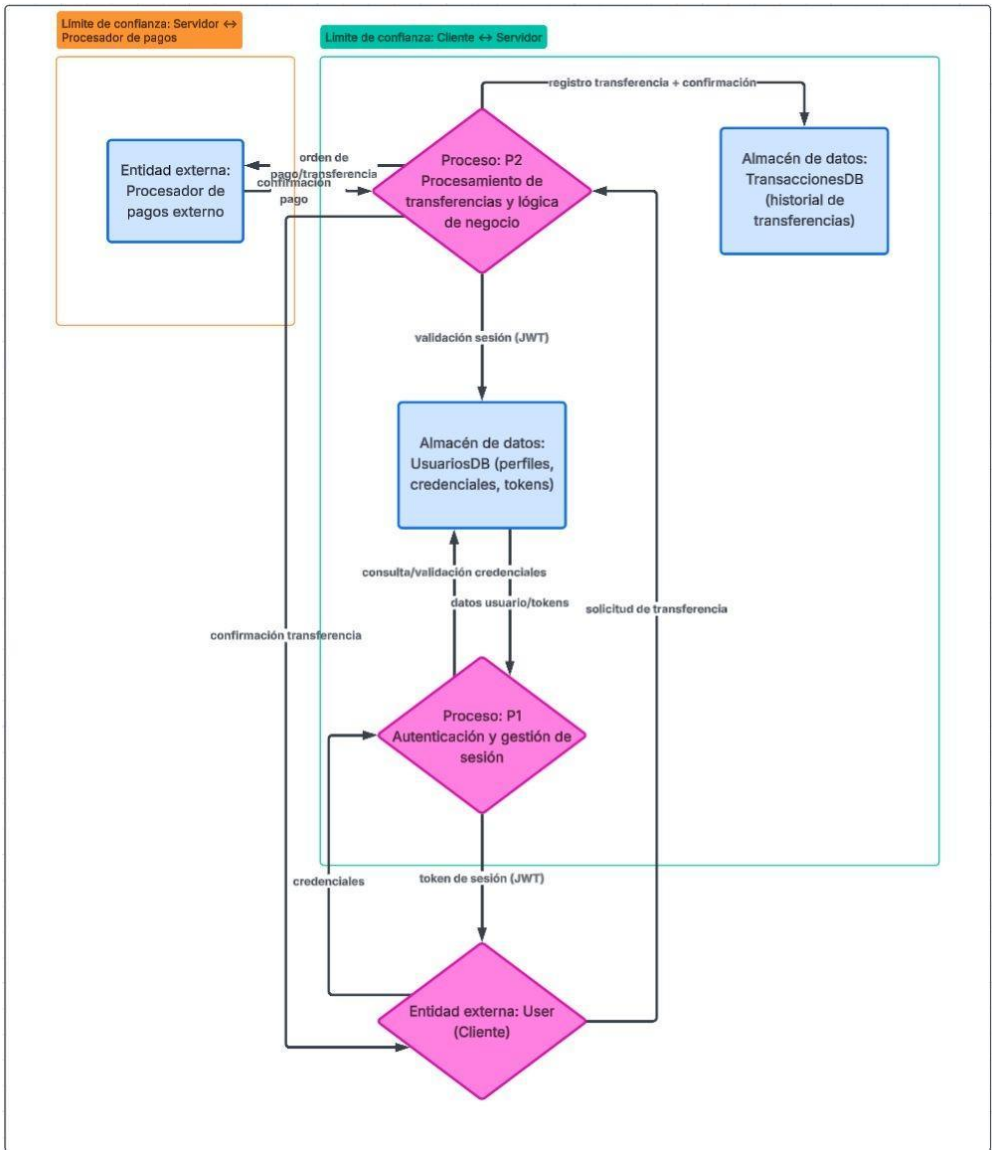
1. Review the base case: online banking web system.

2. In pairs, identify critical assets and list entry points and actors.
   As a team, we identified the following critical components:
   - Frontend web application: A browser-based interface for clients.
   - Backend API Server: Manages authentication, account data, and transactions.
   - Database: Stores user profiles, credentials, and transaction history.
   - Email service: Send alerts and notifications.
   - Admin Portal: Allows bank staff to manage accounts and investigate suspicious activity.

   As a team we identified the following entry points:
   - Web login page.
   - API endpoints for transactions and account data.
   - Admin portal login page.
   - Email notifications (with embedded links).

   As a team, we identified the following actors:
   - **Client: End** user accessing the system.
   - **Bank Administrator:** An employee with privileged access to manage accounts.
   - **Third-party payment processor:** Third-party APIs for interbank transfers.
   - **Attacker (Internal):** Knows the internal processes and has previous physical or logical access.
   - **Attacker (External):** Financial or disruption motivation; attacks via the internet.

3. Create a basic Data Flow Diagram (DFD) with at least:
   - 2 processes, 2 data stores, 2 external entities, 2 data flows with direction.



Part 2: Applying STRIDE to the DFD.

1. Analyze the DFD and review each component, applying STRIDE.
   Before starting we made a recapitulation of the acronym STRIDE, in Spanish it means identity theft, manipulation of data, repudiation, disclosure of information and denial of services, now taking into account this we continue with our analysis:

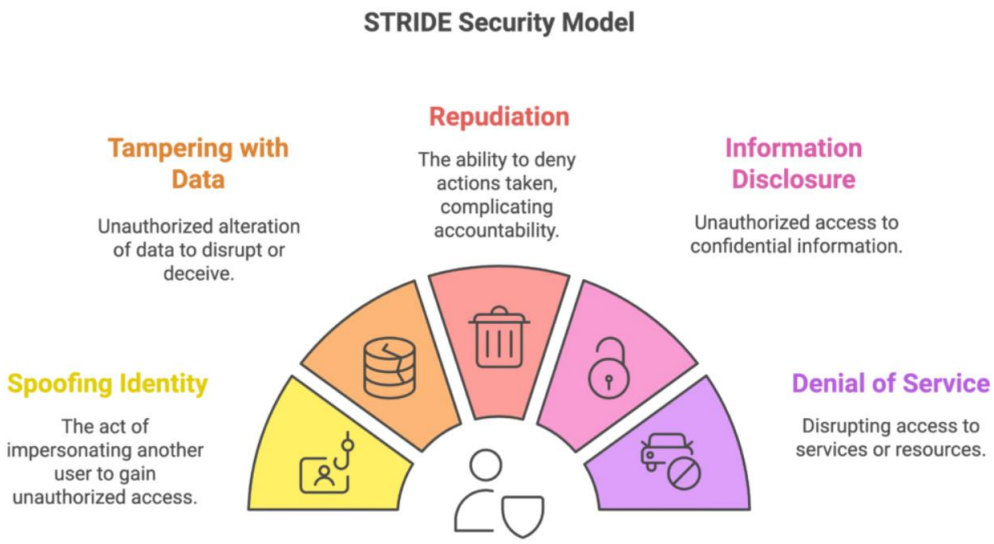


Image 1. Stride Security Model (Taken from https://www.practical-devsecops.com/)

2. Complete a table with at least one threat per component:

Element	Stride Element	Description of the threat	Proposed countermeasure
Login page	Spoofing	Attacker uses *brute force* or *credential stuffing* to impersonate a legitimate customer	Mandatory 2FA Implementation
Towen JWT	Tampering	Modifying payload data to change user ID or privileges.	Signing tokens with robust algorithms, strict signature validation on every request and use of cookies.
Towen JWT	Information Disclosure	The JWT is not encrypted by revealing the user's email or role to anyone.	Do not include sensitive PII in the payload.
Towen JWT	Spoofing	An attacker steals a JWT and uses it to impersonate the user	Short expiry times
API Backend	Information Disclosure	Detailed error messages reveal the	Configuring generic error messages and

		structure of the DB or software versions.	suppressing server headers
API Backend	Tampering	Injection of parameters to alter business logic	Strict validation of schemas and server-side validation logic.
API Backend	Denial of Service	Flood of complex CPU/RAM-consuming requests	Implementation of per-user rate limiting and API usage quotas.
DataBase	Tampering	An internal attacker or via SQL injection modifies balances directly in the database tables.	Use of stored procedures, parameterized queries, and data encryption.
DataBase	Denial of Service	Execution of extremely slow SQL queries that block tables for other users.	Optimization of indexes, query timeouts and monitoring of slow queries.
Admin Portal	Elevation of Privilege	An employee with a "Support" role exploits a logic flaw to gain SuperAdmin permissions	Role-Based Access Control complemented by least privilege.
Admin Portal	Spoofing	Social engineering attack to obtain the credentials of a bank employee.	Authentication for all staff.
Admin Portal	Repudiation	An admin makes a change to an account and deletes the local logs to avoid detection.	Export logs to a centralized, external SIEM system with immutable retention.
Audit Logs	Repudiation	A malicious administrator deletes logs after performing a fraudulent action to leave no trace.	Sending logs in real time to a server.
Transaction API	Tampering	Manipulating parameters in the JSON URL or body	Validation of schemas in the Backend and re-verification of

			balances on the server side before processing.
External Processor	Spoofing	A malicious server impersonates the payment processor to confirm fake transfers.	Webhook signature validation using shared secrets.
Email Service	Information Disclosure	The notification email includes the balance exposing them if the email is intercepted.	Minimizing Data in Notifications
Email Service	Spoofing	The attacker sends fake emails (Phishing)	Strict Implementation of Records
Email Service	Tampering	Man-in-the-Middle intercepts SMTP traffic and changes the commit link.	Force the use of TLS for all communications between mail servers.
API Entry Points	Elevation of Privilege	An attacker uses a common user's token to call functions in the Management Portal.	Validation of "Scope" in JWT tokens and network segmentation for administrative traffic.

Part 3: Risk analysis with DREAD.

1. Select 2-3 threats from the STRIDE analysis.
   As a team we decided to select the following 3 threats:
   - **Tampering in the Transactions API:** Modification of amounts or destination accounts.
   - **Repudiation in the records:** Modification of logs, elimination of evidence in situations of crime.
   - **Tempering in the mail service:** More frequent use in organizations, high reproducibility if there is no organizational culture or user training.

2. Apply the DREAD formula:

```
(D + R + E + A + D) / 5 = Total Risk
```

Threat	Damage	Reproducibility	Exploitability	Affected Users	Discoverability	Average Score
Tampering (API Transactions)	5	4	4	5	3	4.2
Repudiation (Logs/Audit)	4	2	2	5	2	3
Tampering (Email Service)	3	3	3	4	3	3.2

Justification of the score:

- **Tampering in Transaction API (4.2 - CRITICAL).**
  - **Damage (5):** This is the maximum impact. The alteration of destination amounts or accounts results in direct and irreversible financial loss for the bank and customers.
  - **Reproducibility (4):** Once an attacker figures out how to manipulate the JSON of the request, they can easily automate the attack.
  - **Exploitability (4):** High. It does not require physical access or internal privileges; simply intercept or analyze the customer's web/mobile traffic.
  - **Affected Users (5):** Maximum. Any customer who uses the platform to move money is exposed.
  - **Discoverability (3): Medium** . Although the API code is private, the JSON structure is easily deduced by observing network traffic using tools such as Burp Suite.

- **Tampering in Email Service (3.2 - MEDIUM/HIGH).**
  - **Damage (3):** Medium. It does not allow stealing money directly, but it facilitates the theft of future credentials by deceiving the user with false information.
  - **Reproducibility (3): Medium** . It depends on the attacker's ability to be in the path of traffic, which is not always constant.
  - **Exploitability (3):** Medium. Exploiting the SMTP protocol without TLS requires specific tools and a favorable network position.

- **Affected Users (4):** High. The vast majority of customers receive alerts; A manipulated email campaign can affect thousands simultaneously.
- **Discoverability (3): Medium** . Attackers are well aware of how mail servers work and which protocols are often misconfigured.

- **Repudiation in Audit Logs (3.0 - MEDIUM).**
  - **Damage (4):** High. If the logs are deleted, the bank cannot legally defend itself, cannot collect fraud insurance, and cannot identify whether the attacker was an employee.
  - **Reproducibility (2):** Low. Effectively erasing traces without alerting other monitoring systems is difficult to repeat consistently.
  - **Exploitability (2):** Low. It requires administrator-level access to the file system or log database, which is a high barrier to entry.
  - **Affected Users (5):** Maximum. If the logs fail, the integrity of all bank accounts is called into question by a regulatory audit.
  - **Discoverability (2):** Low. An external attacker typically doesn't look for logs first; It's a more common threat in internal administrators who already know where they are.

3. Compare threats and discuss which should be mitigated first
- As a team we decided that the order according to the assigned score should be:
  - **PRIORITY 1: Tampering in Transaction API (4.2).** It is the most dangerous threat because it directly attacks the main asset: money. Mitigation must be immediate (implement digital signatures on requests and cross-validation on the server).
  - **PRIORITY 2: Tampering in Email Service (3.2).** Although it is less financially damaging than the previous one, it is easier for an external attacker to discover. It must be mitigated by forcing security protocols on the mail.
  - **PRIORITY 3: Repudiation in Records (3).** While critical, it's a secondary threat that depends on another attack happening first. The solution is technical (immutability of logs) and can be planned after securing transactions.

The analysis reveals that the absolute priority for the bank should be to secure the Transactions API (4.2), as it combines a devastating financial impact with high ease of execution. While Repudiation (3) and Email Tampering (3.2) are significant risks, they primarily act as facilitators or aggravators.

In terms of security engineering, the strategy should be:

1. Preventive: Shielding the cash flow (API) to avoid immediate impact.
2. Detective: Strengthen the immutability of logs to ensure that any attempt, even if failed, leaves an indelible legal footprint.
3. Communicational: Securing the email channel to protect customer trust and avoid social engineering attacks.

In short, the security of a banking platform is not measured by its strongest component, but by the ability to mitigate those risks where the damage is critical and exploitation is easy.

Part 4: Documentation and export.

1. Export the DFD diagram as .png or PDF.
2. Export STRIDE and DREAD tables.
3. Write a mini report containing: system description, DFD, threats table, DREAD scores, prioritized countermeasures.

Conclusions:

Ivan Santiago Forero Torres:

- As a team, we conclude that it is not enough for the system to be "difficult to hack" at the network level if the business logic is weak. The DREAD analysis showed us that transaction manipulation in the API is the most real danger, because it directly affects customers' money. We learned that even if the connection is secure (HTTPS), if the server does not verify the integrity of each message individually, the system is still vulnerable. The priority should be to protect the transaction itself, not just the path it travels on.

Juan Sebastián Beltrán Rodríguez:

- We identified that one of the greatest dangers does not come from outside, but from the lack of internal control. Upon discovering that audit logs can be deleted. As future engineers, we emphasize that a banking platform is incomplete if it does not guarantee that each action is indelible. Without an immutable audit, the bank has no way to defend itself against fraud by an administrator or an attacker who manages to escalate privileges.

Nicolas Cuervo Naranjo:

- Finally, we determined that the email service is the weakest link in customer communication. If an attacker manages to forge the bank's emails, all the investment in technical security is lost because the user will be easily fooled. We conclude that implementing authentication standards in emails is a simple but mandatory measure to protect the bank's reputation and prevent customers from falling for phishing scams.